AnalogCoder: Analog Circuit Design via Training-Free Code Generation

时间\会议\论文链接

2025 年 AAAI
https://ojs.aaai.org/index.php/AAAI/article/download/32016/34171?utm_source=chatgpt.com

核心技术路线：

让大语言模型生成电路代码，再利用仿真和检查机制发现问题，把错误反馈给模型，反复修正，最终得到满足功能要求的模拟电路。

用户输入电路的设计需求，大语言模型通过 PySpice 等工具描述和构建电路，生成 Python 电路代码，然后执行代码并进行电路仿真，检查代码能否运行，以及电路是否具有预期行为，然后进行多层检查和功能验证，需求、工作点、直流响应和电路功能，最后输出验证的电路，并可将成功的电路存入工具库，供后续设计复用

这里有两个概念：
前向流程：需求 → 生成代码 → 仿真 → 检查。
反馈流程：检查失败 → 分析错误 → 重新生成代码。

技术创新点：

一：AI 生成 Python 而非直接生成 SPICE 网表
普通思路可能是：
自然语言需求 → AI 直接生成 SPICE 网表 → 仿真。
而 AnalogCoder 选择：
自然语言需求 → AI 生成 Python 代码 → 通过 PySpice 构建电路并仿真。
发挥了 LLM 善于编码的能力，而公开训练语料里面 SPICE 电路代码相对较少。所以利用 Python 作为中间表达形式可以发挥模型已有的编程能力。

需要注意的是：Python 只是电路的描述和构建方式，真正判断电路电气行为的仍然是电路仿真器。代码能够运行，不代表电路就一定对。

二：反馈增强设计流程
不是只检查代码有没有语法错误，而是把验证过程划分成四个层次。每一层负责发现不同的问题。

01 需求检查（Requirement Check）

检查电路是否满足基本结构要求，例如输入端和输出端是否存在、要求的器件是否出现，以及器件连接是否符合基本拓扑要求。

02 仿真与静态工作点检查（Simulation and Operating Point Check）

先检查仿真是否能够正常执行，再检查节点静态电压及 MOSFET 的工作状态。例如，对预期处于饱和区的 NMOS，可以检查相应的 GS 和 DS 与阈值电压之间的关系。

03 直流扫描检查（DC Sweep Check）

改变输入端的直流电压，观察输出电压是否产生合理变化。这个过程用于检查输入到输出的信号通路，并辅助寻找合适的偏置电压。

04 功能检查（Function Check）

根据目标电路的类型选择合适的仿真分析，例如 DC、AC 或瞬态分析，检查电路是否实现预期功能。

发现错误后，如何修改？

系统不会仅仅返回“电路失败”，而是把相关错误信息、测试结果和修改提示传回 LLM。模型据此重新生成完整的电路代码，再进入检查流程。

这套机制背后的思想是：

- **模型负责提出设计方案。**
- **仿真器负责提供实际电气行为的证据。**
- **检查规则负责判断哪里不满足要求。**
- **模型根据反馈重新提出方案。**

三：电路工具库

如果我们设计的电路越来越复杂，每次都从零开始生成所有器件与连接，模型任务会越来越困难。



四：三个板块如何配合
这三个模块不是三个互不相关的算法，而是组成一套完整的设计流程：提示让模型更容易提出初始方案，反馈机制提高单个设计任务的成功机会，工具库则帮助系统逐渐积累可复用的电路设计成果。

技术边界

核心思想：Training-Free（无需训练微调）

它主要通过提示工程、外部仿真、反馈修正和工具库来增强现有 LLM 的设计能力，而不是通过重新训练或微调模型获得这种能力。这里的“无需训练”并不意味着不需要电路数据、规则或工程工具。

AnalogCoder 主要解决的是“给定设计任务，如何生成并验证电路”；它并没有真正建立一个能够对任意陌生晶体管级电路进行自下而上结构识别的通用理解模型。